

2025 ICCAD Contest Problem C: Incremental Placement Optimization Beyond Detailed Placement: Simultaneous Gate Sizing, Buffering, and Cell Relocation

Fang, Jia-Lin
NTUEE
B11901003

Cheng, Bo-Yuan
NTUEE
B11901020

Wu, Yi-En
NTUEE
B11901188

Wu, Pin-Yu
NTU
B11502041

I. INTRODUCTION

Incremental placement optimization is a critical step in physical design, aiming to refine Power, Performance, and Area (PPA) after initial placement. Traditional flows apply gate sizing, buffering, and cell relocation in a sequential manner, each followed by time-consuming static timing analysis (STA). This often results in sub-optimal outcomes due to limited global awareness.

The 2025 ICCAD Contest Problem C encourages a unified approach that simultaneously performs gate sizing, buffer insertion, and cell relocation to globally improve timing, power, and wirelength. Solutions must also respect placement legality and runtime constraints.

To address this challenge, we explore gradient-based optimization techniques enabled by GPU acceleration. Inspired by recent works like DREAMPlace [1] and INSTA [2], which leverage deep learning frameworks (e.g., PyTorch) for placement and fast STA, we propose a differentiable framework that integrates physical and timing objectives. This approach enables scalable and accurate optimization across large designs, pushing beyond conventional heuristic methods.

Our report outlines related research, problem formulation, and algorithmic designs toward a high-quality, runtime-efficient solution. Current measurable progress is based on OpenROAD [3]: we deploy a contest-legal adaptive / hybrid timing-repair flow with reproducible evaluation, and identify **hybrid_polish** as our best legal deliverable on `aes_cipher_top`. Differentiable GPU components (INSTA / FusionSizer / RLPlace) remain the roadmap and plug into a Python optimization scaffold.

II. RELATED WORK

A. Wirelength optimization

RLPlace [4] presents a two-stage algorithm for detailed placement in VLSI physical design, leveraging reinforcement learning (RL) for coarse rearrangement and Satisfiability Modulo Theories (SMT) for fine-grain refinement. In stage 1, they use RL to determine optimal swaps of grids within a window

to improve wirelength (HPWL), each window of the layout is partitioned into grids, with an adjacency matrix encoding net connections. In stage 2, they do legalization to solve overlap from stage 1 by SMT method.

In this contest, we should simultaneously optimize timing, power, and wirelength. Therefore, we must extend its function to include timing and power consideration for implementation into our algorithm.

B. Differentiable Gate Sizing

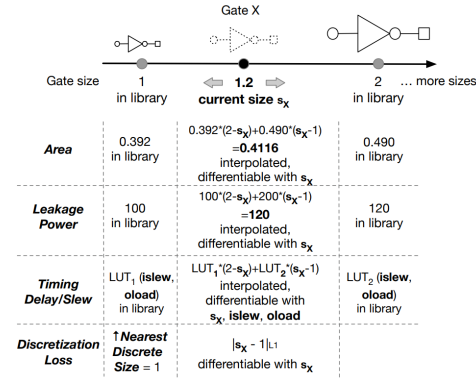


Fig. 1. Differentiable Gate Sizing

FusionSizer [5] proposes a GPU-accelerated framework for large-scale gate sizing, leveraging automatic differentiation to optimize global timing objectives. It reformulates the gate sizing problem as a regression task on gate delay models and performs gradient-based optimization to improve slack across the design.

To make STA differentiable, FusionSizer introduces a fused timing propagation method that approximates maximum and minimum delay using smooth functions such as softmax and softmin. This enables end-to-end differentiability of the timing flow, allowing the optimizer to backpropagate gradients through the STA pipeline.

Moreover, FusionSizer incorporates analytical models for parasitics and delay, which are trained using supervised learning on real Liberty data to achieve high fidelity. It demonstrates significant improvements over commercial tools in both runtime and timing quality, achieving better TNS/WNS with up to 20× speedup.

This differentiable approach to gate sizing complements our contest goals by enabling simultaneous, global timing-driven optimization using scalable GPU resources.

C. GPU acceleration

1) *STA analysis*: INSTA [2] introduces a novel differentiable, GPU-accelerated statistical static timing analysis (STA) engine. Unlike prior GPU-STA efforts that struggle with accuracy due to simplified delay models, INSTA achieves near-perfect correlation (0.999) with industry-standard signoff tools by leveraging a one-time initialization from a reference STA engine (commercial signoff tool).

It splits timing analysis into delay annotation and propagation stages, using CUDA kernels for fast and accurate timing propagation.

Compared with CPU-based STA, it builds a leveled timing graph for the circuit initially, then pass each level to CUDA kernels. This way, the pins in the same level can be calculated in $O(1)$ time for the number of kernels are greater than the number of pins in a single level. INSTA enables gradient computation for global timing metrics (e.g., TNS) with respect to cell locations and gate sizes, making it ideal for large-scale, gradient-based optimization. Applications such as INSTA-Size and INSTA-Place demonstrate superior TNS and wirelength results compared to traditional methods, validating INSTA’s transformative potential in gate sizing and timing-driven placement tasks.

2) *Placer*: DREAMPlace [1] is a GPU-accelerated global placer that formulates placement as a numerical optimization problem. It mimics the analytical placement process using deep learning frameworks (e.g., PyTorch), allowing placement objectives such as wirelength and density to be modeled as differentiable functions. DREAMPlace solves placement using gradient descent, achieving over 10× speedup compared to traditional CPU-based placers. Its integration of GPU-parallelism with differentiable modeling makes it ideal for large-scale designs.

ABCDPlace [6] targets detailed placement by introducing a batched, concurrent approach using multithreaded CPUs and GPUs. It significantly improves the runtime of detailed placement while maintaining quality. ABCDPlace supports GPU-accelerated cell legalization, alignment, and whitespace optimization in a parallel fashion. Unlike traditional detailed placers that sequentially resolve overlaps, ABCDPlace handles batch-based legalizations simultaneously, which scales well for modern designs with millions of instances.

Together, DREAMPlace and ABCDPlace demonstrate how GPU acceleration can be effectively applied to both global

and detailed placement. Their techniques inspire our work in extending placement optimization with timing-awareness, resizing, and buffering under modern compute architectures.

III. PROBLEM FORMULATION

The objective is to enhance Power, Performance, and Area (PPA) of an initial placement by optimizing:

- **Cell Relocation**: Movement of standard cells and macros.
- **Gate Sizing**: Selection of alternate drive strengths.
- **Buffer Insertion**: Minimizing delay on critical nets.

A. Scoring Metrics

The overall score S is computed using the following components:

$$S = 1000 \times P - 50 \times D - 300 \times R \quad (1)$$

where:

- P measures PPA improvement:

$$P = \alpha \times TNS_{norm} + \beta \times POWER_{norm} + \gamma \times WL_{norm} \quad (2)$$

- D is the averaged displacement penalty:

$$D = \frac{\sum \text{Manhattan displacement}}{\text{Total number of cells}} \quad (3)$$

- R accounts for runtime efficiency.

The weighting factors α, β, γ vary per testcase.

B. Legal Constraints

- Cells must be positioned on valid placement sites.
- No overlaps between cells.
- IO ports and fixed macros must remain unchanged.
- Library consistency must be maintained.

IV. ALGORITHM

We utilize OpenROAD [3], an open-source digital design automation tool that offers a comprehensive suite of functionalities extending far beyond simple evaluation. OpenROAD [3] enables not only placement and routing analysis but also advanced timing optimization and power estimation. To improve the performance of the circuit, we applied a sequence of optimization steps centered around timing repair. Specifically, we first perform placement-level parasitic estimation to get an initial view of delay distribution. Then, we invoke *repair_timing* with the *-setup* flag, which attempts to fix timing violations related to setup slack by selectively resizing cells, inserting buffers, and adjusting placement, while explicitly skipping disruptive transformations like pin swapping, gate cloning, and buffer removal which is not a legal move limited by the contest rules. These steps are combined with detailed placement refinement after each repair iteration to ensure legal and optimized placement. This iterative approach improves circuit timing metrics such as Total Negative Slack (TNS) and Worst Negative Slack (WNS). Furthermore, we do power, runtime and wirelength evaluation at the end of the optimizing process to calculate the improvement at the end.

Beyond one-shot *repair_timing*, we wrap OpenROAD with an **adaptive multi-strategy driver** and a **hybrid** flow that (i) selects or composes resize / buffer / legalize / improve passes from baseline TNS pressure, (ii) uses only contest-legal flags (*-skip_pin_swap*, *-skip_gate_cloning*, *-skip_buffer_removal*), (iii) legalizes after disruptive passes, and (iv) optionally recovers power and polishes wirelength. Metrics (TNS, WNS, power, HPWL, average displacement D , runtime) are logged under `openRoad_eval_script/results/` for ranking.

A. Timing - Greedy Algorithm

The algorithm (Algorithm 1) we apply to optimize the circuit is basically greedy method. It is designed to attempt timing repairs along a given path with setup violations of the given initial placement. We try to solve the timing in advanced because a negative TNS could make the circuit illegal when implementation.

First, it expands the path to analyze each gate along it and calculates the load-dependent delay for every gate driver. It then sorts these gates in descending order of their load delays, prioritizing those with the highest impact on timing. The algorithm determines how many repair actions (such as resizing, buffer insertion, or pin swapping) should be attempted in this pass, scaling the effort based on how severe the timing violation is. For each prioritized gate, it iteratively tries a sequence of repair moves, stopping at the first successful change. The process is limited by a dynamically calculated number of allowed repairs per pass to balance between aggressive fixing and over-correction. If any repair is successfully applied to the path, the algorithm returns true, indicating progress; otherwise, it returns false. This approach allows the repair process to focus on the most problematic points in the path and adapt its aggressiveness based on the overall timing situation.

B. Adaptive strategies and hybrid flow (deployed)

Building on Algorithm 1, we run several legal OpenROAD wrappers and a dedicated hybrid driver:

- **timing_first**: detailed placement $\rightarrow$ buffer+resize repair $\rightarrow$ legalize $\rightarrow$ improve_placement;
- **size_only**: resize-only repair (*-skip_buffering*) + legalize + improve (lower HPWL / displacement when WL is costly);
- **full_buffer**: buffer+resize without pre-DP, then legalize + improve;
- **adaptive**: resize-first, then optional capped buffering if $|TNS|$ remains large, then improve + power recovery;
- **hybrid** (Algorithm 2): resize-only pass $\rightarrow$ limited buffering only if $|TNS|$ exceeds a threshold $\rightarrow$ improve + recover_power $\rightarrow$ legalize;
- **polish**: load an existing legal DEF and apply improve_placement (+ optional light power recovery) without a heavy full re-repair.

Our strengthen pipeline runs hybrid from the original DEF, then polishes the strongest sweep seeds (*full_buffer*,

Algorithm 1 Repair Setup Path Pseudo Code (From OpenROAD [3])

Require: Path pointer *path*, setup slack *s*, slack margin ϵ

Ensure: true iff at least one repair is applied

```

1:  $E \leftarrow \text{PathExpanded}(path)$   $\triangleright$  Get detailed access to
   every element on that path
2: if  $|E| \leq 1$  then
3:   return false  $\triangleright$  No repair is done
4: end if
5:  $L \leftarrow []$   $\triangleright$  (index, load-delay) pairs
6:  $start \leftarrow E.startIndex()$ 
7:  $dcalc\_ap \leftarrow path.dcalcAnalysisPt()$ 
8:  $lib\_ap \leftarrow dcalc\_ap.libertyIndex()$ 
9: for  $i \leftarrow start$  to  $|E| - 1$  do  $\triangleright$  Find delay for every gate
10:   $p_i \leftarrow E.path(i)$ 
11:   $pin \leftarrow p_i.pin()$ 
12:  if  $i > 0$  and  $isDriver(pin)$  and
      $\neg isTopLevelPort(pin)$  then
13:     $prev\_arc \leftarrow p_i.prevArc()$ 
14:     $corner \leftarrow prev\_arc.cornerArc(lib\_ap)$ 
15:     $prev\_edge \leftarrow p_i.prevEdge()$ 
16:     $delay \leftarrow arcDelay(prev\_edge, prev\_arc) -$ 
        $intrinsicDelay(corner)$ 
17:    Append  $(i, delay)$  to  $L$ 
18:  end if
19: end for
20: Sort  $L$  descending by  $delay$  (breaking ties by larger  $i$ )
21:  $r \leftarrow 1$   $\triangleright$  Repairs allowed this pass
22: if  $max\_viol - min\_viol \neq 0$  then
23:    $r \leftarrow r + \text{round}\left((R_{max} - 1) \cdot \frac{-slack - min\_viol}{max\_viol - min\_viol}\right)$ 
24: end if
25: if fallback then  $\triangleright$  No improve after moves
26:    $r \leftarrow 1$ 
27: end if
28:  $changed \leftarrow 0$ 
29: for all  $(idx, \_)$  in  $L$  do
30:   if  $changed \geq r$  then
31:     break
32:   end if
33:    $drv\_path \leftarrow E.path(idx)$ 
34:   for all  $m \in \text{move\_sequence}$  do
35:     if  $m.doMove(drv\_path, idx, s, E, \epsilon)$  then
36:        $changed \leftarrow changed + 1$ 
37:       break  $\triangleright$  Next driver
38:     end if
39:   end for
40: end for
41: return  $changed > 0$ 

```

size_only, and hybrid). Among all legal outputs, **hybrid_polish** achieves the best TNS and WNS on aes_cipher_top.

Algorithm 2 Hybrid Contest-Legal OpenROAD Driver

```

1: Load Liberty/LEF/DEF/SDC; snapshot cell locations for
   displacement  $D$ 
2: estimate_parasitics -placement;      measure
   baseline TNS
3: Pass 1:      repair_timing -setup      with
   -skip_buffering (legal flags)
4: Legalize (detailed_placement)
5: if |TNS| >  $\tau$  then
6:   Pass 2:      buffered      repair      with
   -max_buffer_percent cap
7:   Legalize
8: end if
9: improve_placement; optional -recover_power;
   legalize
10: Report PPA; write DEF; record runtime and average
   Manhattan  $D$ 

```

C. Proposed algorithm in future work

Algorithm 3 shows our plan that including the method provided the related works, aiming to do some integration and extension to apply to this contest. We propose a window-based incremental optimization framework that iteratively refines placement and sizing decisions to improve design quality under timing and congestion constraints. The method begins by initializing key parameters, including the initial window size w , its maximum limit $w_{\max}$, an increment value w_{inc} , and a convergence threshold defined by the number of consecutive non-improving iterations rot . The baseline cost is evaluated using the original placement .def file provided (or the best OpenROAD seed such as hybrid_polish). At each window size, the algorithm performs a loop where reinforcement learning (RL)-based (or may turn to use other methods such as ABCDPlace if RL is not stable and efficient enough) cell reallocation and analytical gate sizing are applied, followed by static timing analysis (STA) using the INSTA engine, which also updates the congestion parameters. Buffering is preferred only under a budget after resize/relocate. The cost of the modified design is then evaluated under a contest-aligned proxy of S . If the new cost improves upon the current best, it is recorded and the non-improvement counter is reset; otherwise, the counter is incremented. This process continues until no further improvements are observed for rot iterations, after which the window size is increased by w_{inc} . The algorithm terminates when the maximum window size is reached, and the changelist corresponding to the lowest cost configuration is returned as the final output. A Python scaffold under src/python/framework/ encodes this accept/reject loop with plug-in STA / size / buffer / relocate backends.

Algorithm 3 Incremental Window-Based Optimization Routine with RL

```

1: Initialization:
2: Set window size  $w$ 
3: Set max window size  $w_{\max}$ 
4: Set window increment value  $w_{\text{inc}}$ 
5: Set required optimizing threshold  $rot$ 
6: Set no-improvement counter  $n_{\text{improv}}$ 
7: Set displacement budget  $D_{\max}$  and buffer fraction cap  $b_{\max}$ 
8:  $best\_cost \leftarrow \text{cost\_eval}(\text{OpenROAD seed .def}, \text{exec\_time} = 0)$ 

9: Optimizing routine:
10: while  $w < w_{\max}$  do
11:    $n_{\text{improv}} \leftarrow 0$ 
12:    $congestion\_param \leftarrow \text{zero\_vector}$ 
13:   while  $n_{\text{improv}} < rot$  do
14:     Reallocate cells with RL(window size =  $w$ , congestion parameters) under  $D_{\max}$ 
15:     Analytical Gate sizing (congestion parameters)
16:     Optionally buffer critical nets with fraction  $\leq b_{\max}$ 
17:     Perform STA; update  $congestion\_param \leftarrow \text{INSTA}(\text{window size} = w)$ 
18:      $new\_cost \leftarrow \text{cost\_eval}(\text{new .def}, \text{exec\_time}, \text{STA})$ 
19:     if  $new\_cost < best\_cost$  and placement legal
20:        $n_{\text{improv}} \leftarrow 0$ 
21:        $best\_cost \leftarrow new\_cost$ 
22:     else
23:        $n_{\text{improv}} \leftarrow n_{\text{improv}} + 1$ 
24:     end if
25:   end while
26:    $w \leftarrow w + w_{\text{inc}}$ 
27: end while
28: Output: changelist of lowest cost

```

V. EXPERIMENTAL RESULTS

We conducted experiments based on the OpenROAD [3] flow to evaluate various combinations of timing repair strategies, including gate sizing, buffer insertion, and detailed placement legalization. Table I summarizes the impact on Total Negative Slack (TNS), Worst Negative Slack (WNS), total power, half-perimeter wirelength (HPWL), and runtime.

The baseline design exhibits a significant timing violation with TNS of -100709.3 and WNS of -339.64, serving as the reference for improvement metrics. Applying `repair_timing -setup` without any legality constraints achieves the best TNS and WNS (-11687.35 and -45.12 respectively), but the result is not placement-legal.

When enforcing legal movements while allowing both buffering and resizing (`repair_timing -setup -skip_pin_swap -skip_gate_cloning`), the TNS slightly worsens to -12185.75 and WNS to -46.33, though

TABLE I
COMPARISON OF TIMING REPAIR STRATEGIES

Method	TNS	TNS Impr. (%)	WNS	Power (W)	HPWL	Runtime (s)
OpenROAD [3] v2.0-17598-ga008522d8						
Baseline	-100709.3	0.00	-339.64	5.53E-02	32394.3u	8.463
Repair (Not Legal)	-11687.35	-88.39	-45.12	5.80E-02	–	93.272
Buffer + Resize (Not Legal)	-12185.75	-87.90	-46.33	5.77E-02	–	71.9
Buffer + Resize + DP	-15592.41	-84.52	-81.69	5.91E-02	40027.2u	71.237
Only Resize (Not Legal)	-13444.71	-86.65	-57.79	5.71E-02	–	124.416
Only Resize + DP	-15946.53	-84.17	-72.10	5.77E-02	35290.9u	121.441
DP + Buffer + Resize + DP	-14786.6	-85.32	-85.95	5.92E-02	44278.3u	79.852
OpenROAD [3] v2.0-22086-g1f25bd6856						
DP + Buffer + Resize + DP + Improved placement	-12829.12	-87.26	-71.20	5.75E-02	34396.8u	62.826

the result require further detailed placement legalization as now there may be some overlap, but runtime improves significantly. Incorporating detailed placement legalization increases the TNS to -15592.41 and WNS to -81.69, with an average cell displacement of $0.6\mu\text{m}$ and HPWL penalty of 20%. This represents a better tradeoff between timing recovery and physical legality.

To evaluate the isolated impact of gate resizing, we disabled buffer insertion via the `-skip_buffering` flag. In this case, resizing alone with detailed placement recovers TNS to -15946.53 and WNS to -72.10, with only a 9% increase in HPWL and a low average displacement of $0.3\mu\text{m}$.

Overall, buffer insertion proves effective in reducing TNS aggressively. However, the cost on wirelength is higher as it disturbs the original placement. Also, without buffering, only allows gate resizing is very time consuming as more iteration is needed and more gate should be considered to reduce the delay. Detailed placement is needed for giving legalized output. We only provide HPWL with legalized case.

Finally, we apply detailed in advanced and this can further decrease the TNS penalty (TNS = -14786.6). Meanwhile, the cost on wirelength is huge. Also, adding improved placement at the end could further reduce wirelength.

Notice that we found that different OpenROAD [3] version could lead to different results, the experiment is conducted in OpenROAD [3] v2.0-17598-ga008522d8 and v2.0-22086-g1f25bd6856.

A. Extended sweep, hybrid, and polish (OpenROAD v2.0-17598)

Table II reports our reproducible strategy sweep and strengthen pipeline on the same `aes_cipher_top` design (OpenROAD v2.0-17598-ga008522d8). Displacement D is the average Manhattan distance versus the original DEF (microns). Runtime for `*polishrowsispolish` – `onlywalltimeandshouldnotbecompareddirectlyagainstfulloptimiz`

Key observations from Table II:

- **hybrid_polish is the best legal result:** TNS -8792.6 and WNS -47.79, clearly better than early legal

recipes in Table I and better than other sweep/strengthen variants on timing.

- Aggressive buffering (e.g., adaptive) can improve TNS versus baseline but inflates HPWL and D , hurting the overall contest tradeoff.
- `size_only` achieves strong TNS with low HPWL but is slower; `full_buffer` is a strong balanced seed.
- Polishing `full_buffer` without care can worsen TNS (power recovery / local moves); polishing the hybrid seed is what yields the best timing.
- A further polish of `hybrid_polish` with power recovery disabled (`hybrid_polish_nr`) does not beat `hybrid_polish` on TNS.

The recommended deliverable DEF is `aes_cipher_top_hybrid_polish.def`.

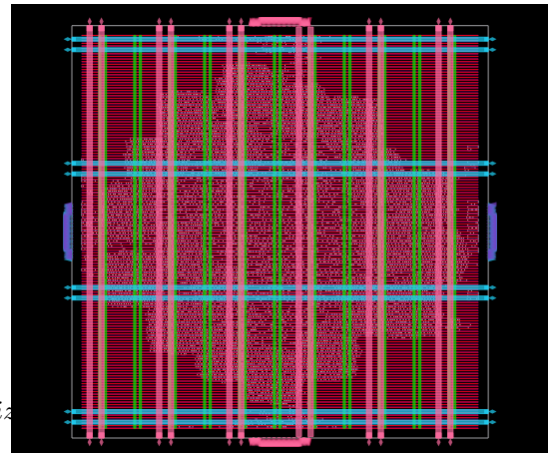


Fig. 2. layout of the baseline

TABLE II
EXTENDED LEGAL STRATEGIES ON AES_CIPHER_TOP (OPENROAD v2.0-17598)

Method	TNS	TNS Impr. (%)	WNS	Power (W)	HPWL	D (μm)
Baseline	-100709.3	0.00	-339.64	5.53E-02	32394.3u	0.00
size_only	-12005.4	88.08	-73.81	5.71E-02	32564.7u	0.69
full_buffer	-14308.0	85.79	-61.91	5.60E-02	34178.2u	0.66
timing_first	-17188.0	82.93	-66.82	5.60E-02	34429.5u	0.69
adaptive	-13076.5	87.02	-77.72	5.80E-02	38525.1u	1.30
hybrid	-15690.4	84.42	-61.68	5.73E-02	35837.1u	0.95
full_buffer_polish	-17726.3	82.40	-66.51	5.76E-02	33756.8u	0.66
size_only_polish	-16469.7	83.65	-82.45	5.77E-02	33920.2u	0.68
hybrid_polish (best)	-8792.6	91.27	-47.79	5.98E-02	35082.3u	0.95
hybrid_polish_nr	-8809.9	91.25	-47.23	5.97E-02	—	0.96

VI. CONCLUSION

Earlier manual OpenROAD experiments suggested that decreasing TNS is helped by detailed placement first, then greedily inserting buffers and upsizing gates on high-delay paths, then detailed placement again for legalization, with improve_placement further lowering wirelength cost. Initial detailed placement can reduce TNS penalty but may increase wirelength; buffering also tends to inflate HPWL because new cells are inserted.

With the extended sweep and strengthen pipeline, we refine that conclusion: **hybrid_polish is our best legal result** on aes_cipher_top (TNS -8792.6, WNS -47.79), produced by a resize-first hybrid repair with capped buffering, followed by polish. For high TNS penalty we still favor early legalization / resize-heavy passes and avoid unbounded buffering when wirelength or displacement pressure is high. Contest ranking should emphasize TNS/WNS/HPWL/ D jointly; polish-only runtimes must not be mistaken for full optimize runtimes when estimating R .

VII. FUTURE WORKS

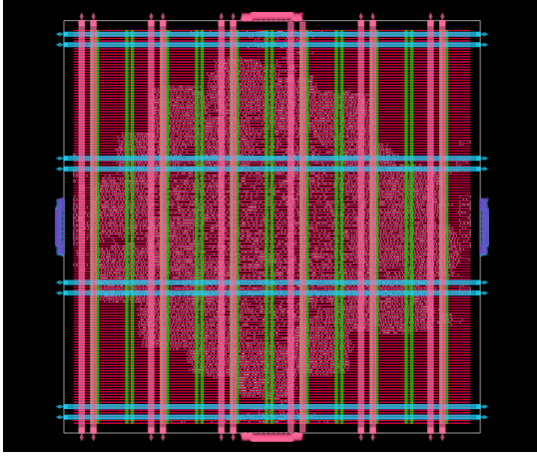


Fig. 3. layout of repair timing but not legal

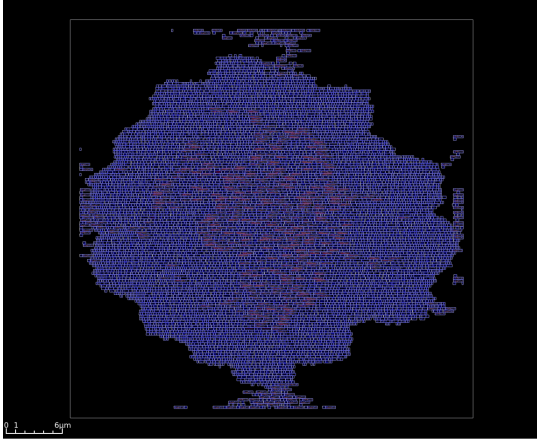


Fig. 4. Layout of repair timing and legal, disable nets (including initial detailed placement)

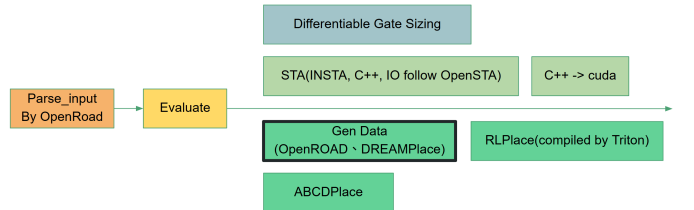


Fig. 5. Future work timeline

A. STA

We mainly want to use the method of INSTA [2], taking advantage of its GPU speed up. We expect to test it in c++ or python and then implement it in CUDA. Also, the back propagation in INSTA's [2] algorithm could tell us the congestion spot for optimize, making a good parameters for determining cells to relocate, resize and place to insert buffer. The existing Python sta_fn hook should be bridged before full CUDA kernels.

B. Gate sizing

The analytical method require differentiable function, and interpolation method [5] provided us a good directions for realization this method.

C. Relocation

We are going to extend the RLplace [4] method to consider wirelength (originally only), timing and power. Our training material are mainly from generation of other placer [1] with original verilog file. However, possible runtime and stableness problem may happens and we aim to solve this problem. We plan to do the runtime test, and decide whether to use the method of ABCDPlace [6] (with GPU acceleration) instead.

D. Engineering

Keep the OpenROAD hybrid / polish flow as the legal seed and fallback evaluator; use ICCAD_ProbC_ENV (CUDA + PyTorch) for differentiable backends; regress metrics with compare_results.py on every strategy sweep.

VIII. JOB DIVISION

A. Fang, Jia-Lin

Responsible for reviewing related papers, conducting experiments to identify the combination with the largest slack, and designing the algorithm.

B. Cheng, Bo-Yuan

Responsible for reviewing related papers, conducting experiments to identify the combination with the largest slack, and writing the report.

C. Wu, Yi-En

Responsible for reviewing related papers, conducting experiments to identify the combination with the largest slack, and writing the report.

D. Wu, Pin-Yu

Responsible for reviewing related papers; setting up the project-local OpenROAD Docker workflow; implementing and running the adaptive / hybrid / polish strengthen pipeline; ranking legal strategies and selecting hybrid_polish as the best deliverable; updating the experimental results and conclusion in the report; giving the oral presentation; and preparing the slides.

REFERENCES

- [1] Yibo Lin, Zixuan Jiang, Jiaqi Gu, Wuxi Li, Shounak Dhar, Haoxing Ren, Brucek Khailany, and David Z. Pan. Dreamplace: Deep learning toolkit-enabled gpu acceleration for modern vlsi placement. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, 40(4):748--761, 2021.
- [2] Yi-Chen Lu, Zhizheng Guo, Kishor Kunal, Rongjian Liang, and Haoxing Ren. Insta: An ultra-fast, differentiable, statistical static timing analysis engine for industrial physical design applications. In *2025 62th ACM/IEEE Design Automation Conference (DAC)*, 2025.
- [3] Tutu Ajayi, David Blaauw, Tuck-Boon Chan, C.-K. Cheng, Vidya A. Chhabria, D. K. Choo, M. Coltella, Ronald G. Dreslinski, Mateus Fogaca, S. Mehdi Hashemi, A. A. Ibrahim, Andrew B. Kahng, M. Kim, J. Li, Z. Liang, Uday Mallappa, Paul I. Pérez, Geraldo Pradipta, Sherief Reda, Austin Rovinski, Kambiz Samadi, Sachin S. Sapatnekar, Lawrence K. Saul, Carl Sechen, Vaishnav Srinivas, William Swartz, Dennis Sylvester, D. Urquhart, L Wang, Mingyu Woo, and B. Xu. Openroad: Toward a self-driving, open-source digital layout implementation tool chain. 2019.
- [4] Uday Mallappa, Sreedhar Pratty, and David Brown. Rlplace: Deep rl guided heuristics for detailed placement optimization. In *2022 Design, Automation Test in Europe Conference Exhibition (DATE)*, pages 120--123, 2022.
- [5] Yufan Du, Zizheng Guo, Yibo Lin, Runsheng Wang, and Ru Huang. Fusion of global placement and gate sizing with differentiable optimization. In *2024 IEEE/ACM International Conference on Computer-Aided Design (ICCAD)*. ACM, 2024.
- [6] Yibo Lin, Wuxi Li, Jiaqi Gu, Haoxing Ren, Brucek Khailany, and David Z. Pan. Abcdplace: Accelerated batch-based concurrent detailed placement on multithreaded cpus and gpus. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, 39(12):5083--5096, 2020.